Revision 14-17 · The KV Cache Story

Interactive revision of lessons 14, 15, 16 and 17: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · How much work does the naive loop waste? PARTIAL

Covers: 14

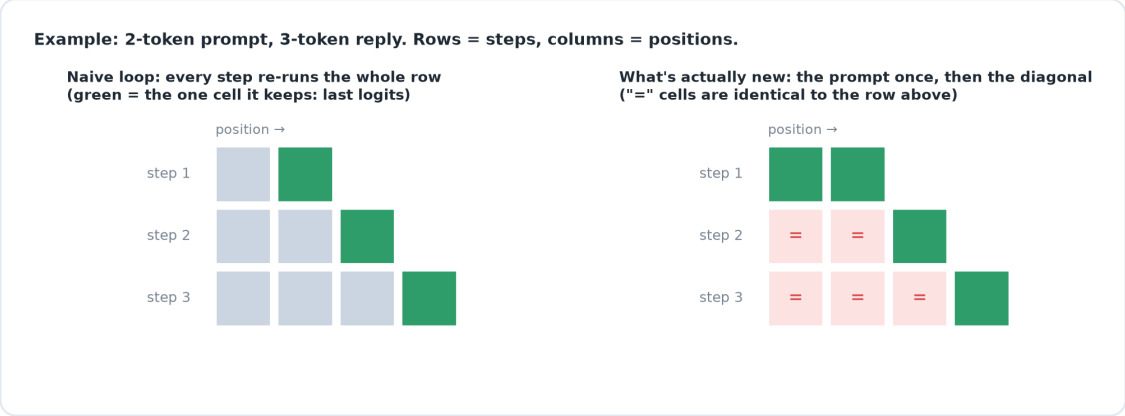
QUESTION

To generate a reply, the model predicts one token at a time. The naive way re-feeds the **entire sequence so far** on every step. With a 512-token prompt and a 256-token reply: how many tokens does it push through the model in total, how many are actually needed, what's the waste ratio, and why is the rest unnecessary?

First, a tiny example (prompt P = 2 tokens, reply N = 3 tokens). Each step re-feeds everything so far and keeps only the last position's prediction:

- Step 1: feed 2 tokens → predict token #3. **Processed 2.**
- Step 2: feed 3 tokens (prompt + 1 generated) → predict #4. **Processed 3.**
- Step 3: feed 4 tokens → predict #5. **Processed 4.**

Total processed = 2 + 3 + 4 = 9, but only 5 tokens exist. The grid shows where the waste is: each step recomputes the whole row, but only the green diagonal cell is new; every "=" cell is identical to the row above.



Now the real numbers (P = 512, N = 256). Step k feeds 512 + k tokens, for k = 0 … 255:

1. The **prompt** (512 tokens) is re-fed on all 256 steps: 256 × 512 = **131,072**.
2. The **growing reply** adds 0, then 1, … up to 255: that sum is 255 × 256 ÷ 2 = **32,640**.
3. Total ≈ **163,712** tokens pushed through the model.
4. **Needed:** each of the 768 real tokens (512 + 256) goes through the model **once** to compute its key and value: P + N = **768**.
5. Waste ratio: 163,712 ÷ 768 ≈ **213×** (roughly the reply length).

6. **Why unnecessary:** under the causal mask, token #5's key and value depend only on tokens 1–5, so adding token #6 later can't change them. The naive loop recomputes the same keys/values every step. Fix: compute each token's K and V once and store them (the KV cache); each step then does the work for just the one new token.

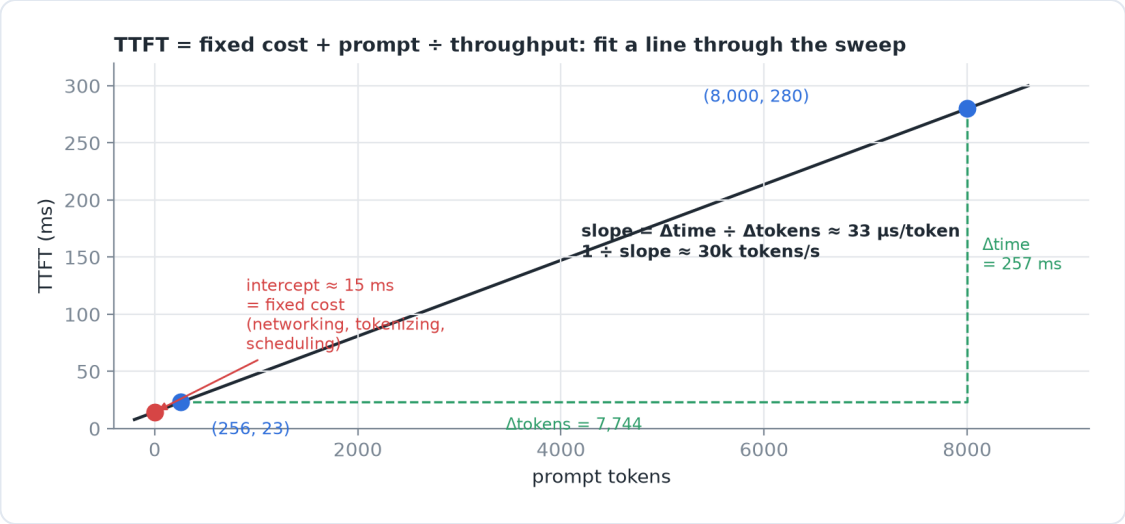
> **RE-DRILL**
>
> Separate "what a step uses" from "what must be computed at least once".

Q2 · Read prefill throughput off a TTFT sweep MISSED

Covers: 15

> **QUESTION**
>
> TTFT (time to first token) is measured one request at a time, max_tokens = 1, taking medians. 256-token prompt → 23 ms; 8,000-token prompt → 280 ms. Model TTFT as a straight line, **TTFT = fixed cost + prompt ÷ throughput**. What are the prefill throughput and the fixed cost, and what is the fixed cost made of?

Why it's a line. Every prompt pays the same one-off overhead (the **intercept**), then an amount that grows with the number of tokens (the **slope**). Two measurements give two points, and a line through them has a slope and an intercept.



How to solve it

1. **Slope** = Δtime ÷ Δtokens = (280 − 23) ms ÷ (8,000 − 256) = 257 ms ÷ 7,744 ≈ **0.033 ms/token** (33 μs). Taking the *difference* of the two points cancels the fixed cost, leaving only the per-token part.
2. **Throughput** = 1 ÷ slope = 1 ÷ 0.033 ms ≈ **30,000 tokens/s**, matching the napkin "half of H100 peak" (lesson 15).
3. **Fixed cost (intercept)** = take either point and subtract its token part: 23 − 256 × 0.033 ≈ 23 − 8.5 ≈ **14.5 ms**.

4. **Made of:** per-request overhead (**networking, tokenization, scheduling**), not prefill (that's the slope) and not decode (max_tokens = 1, so there is none).

> **RE-DRILL**
>
> With a fixed cost in play, use the difference between two points, never a single point's ratio.

Q3 · TPOT from timestamps, and a stuttering stream PARTIAL

Covers: 16, 15

> **QUESTION**
>
> One request: TTFT 0.3 s, E2E 2.3 s, 401 output tokens. What's TPOT, and why not E2E ÷ 401? Average TPOT looks fine but users say the stream stutters: what do you look at?

Answer

- **TPOT = (E2E − TTFT) ÷ (N_out − 1)** = (2.3 − 0.3) ÷ 400 = **5 ms**. E2E ÷ 401 would fold the prefill pause (TTFT) into the per-token pace; and the first token arrived with TTFT, so only 400 gaps remain.
- **Stutter:** look at the **individual inter-token gaps** and their tail (P95/P99), not the average. A common cause: another user's big prefill occupies the GPU and every stream pauses behind it (lesson 15); fix with chunked prefill.

Q4 · Three KV cache bugs MISSED

Covers: 17, 11

> **QUESTION**
>
> A preallocated KV cache shows: (1) first token fine, the rest gibberish; (2) outputs subtly worse, buffer preallocated with zeros and attention reads all of it; (3) an answer mentions the previous user's conversation. The bug behind each?

Answer

1. **RoPE position:** queries and keys are rotated by their position. The decode token was fed as position 0 instead of the pointer's position (e.g. 512). The first token comes from prefill (positions right), every later step uses the wrong rotation → gibberish.
2. **Reading empty slots:** a zero key scores $q \cdot 0 = 0$, and $e^0 = 1$, so every unfilled slot steals attention weight. Fix: slice or mask to the filled length.
3. **No reset between requests:** the pointer and old K/V weren't cleared, so the new user attends to the previous user's text. Fix: reset per request. A privacy bug, not just quality.

RE-DRILL

Match the symptom's timing: "first token fine, then broken" points at something that differs between prefill and decode.

Exercise · Toy KV cache vs naive recompute SOLID

Covers: 14, 17

QUESTION

One attention head on random 8-number vectors. Write the cached loop (preallocated K/V buffers, a position pointer, read only filled slots) and prove it matches the naive loop exactly; then read the empty slots to see bug 2.

numpy, demystified. Only two ideas appear. (1) A **token is 8 numbers** (a vector); stacking n of them makes an `n × 8` table. (2) `@` is **multiply-and-add**, with one rule: `(rows × INNER) @ (INNER × cols) = (rows × cols)`; the matching INNER cancels and is the thing you sum over.

- `x[t] @ Wq` : one token (8) times an 8×8 matrix → its query (8 numbers). Same for the key (`Wk`) and value (`Wv`).
- `K @ q` : keys `n × 8` times query `8` (a column on the right) → **n scores**, one per key. The 8s cancel, so each score is one key dotted with the query.
- `w @ V` : weights `n` (a row on the left) times values `n × 8` → **one blended value** (8 numbers). The ns cancel, so it's the weighted average of the value rows. (`V @ w` would not line up, and would not mean "blend the rows".)
- A 1-D array has no orientation: on the **left** of `@` it acts as a row, on the **right** as a column.

Slicing is just picking rows: `x[:prompt_len]` is "the first `prompt_len` tokens", `K_cache[pos] = …` writes one row, `K_cache[:n]` reads the first n. `np.zeros((32, 8))` is a 32×8 table of zeros; `np.allclose(a, b)` means "equal within rounding".

`attend(q, K, V)` is the whole attention formula in three lines: `scores = K @ q / √d` (match query to every key), `w = softmax(scores)` (turn scores into weights that sum to 1), `return w @ V` (blend the values by those weights).

```
def cached_outputs(x, prompt_len, read_all_slots=False):
    K_cache = np.zeros((max_len, d))                # 1 · preallocated, fixed shape, zeros
    V_cache = np.zeros((max_len, d))
    pos = 0
    K_cache[:prompt_len] = x[:prompt_len] @ Wk       # 2 · prefill: whole prompt in one shot
    V_cache[:prompt_len] = x[:prompt_len] @ Wv
    pos = prompt_len

    def read():                                      # 4 · filled rows only (bug flag: all rows)
        n = max_len if read_all_slots else pos
        return K_cache[:n], V_cache[:n]

    outs = [attend(x[prompt_len - 1] @ Wq, *read())]   # first output comes from prefill
    for t in range(prompt_len, len(x)):              # 3 · decode: one k, v per new token
        K_cache[pos] = x[t] @ Wk
        V_cache[pos] = x[t] @ Wv
        pos += 1
        outs.append(attend(x[t] @ Wq, *read()))      # query used once, never cached
    return np.array(outs)
```

Result: `cached == naive: True`; reading the zero-filled slots gives a max error of **~3.97**, because each zero key scores 0 and gets weight $e^0 = 1$.